COMPLETE FUNCTION CATALOG

HORIZON GRID

Function Reference

Version: 0.2.4
Documentation prepared: 2026-08-21

PREPARED FOR EVALUATION

Table of Contents

Standalone Function Reference 3

Standalone Function Reference

This chapter is the exhaustive, page-by-page catalog of every major user-facing function, button, and action in HORIZON GRID. Where the Architecture, API Reference, and User Manual chapters explain *why* the platform is built the way it is and walk through representative examples, this chapter is a lookup table: for a given screen, what can you click, who is allowed to click it, what happens when you do, and what happens when it goes wrong.

Every entry below was traced to a real source file -- the page component under `frontend/app/**/page.tsx`, the component it renders from `frontend/components/dashboard/`, the exact function it calls in `frontend/lib/api.ts`, and the backend route (and `require_permission(...)` check) that function actually hits. Nothing here is inferred from the product's marketing description; if a button exists in this chapter, it exists in the running application.

How to Read This Reference

Every function below follows the same nine-field template:

Field	Meaning
Location	The route (page) the function lives on, and the component that renders it.
Purpose	What it does and why it exists.
Role Required	The RBAC role(s) that can actually complete the action -- traced to the real `require_permission("<permission>")` dependency on the backend route the frontend calls, not to what the button's visibility alone might suggest. HORIZON GRID has exactly three roles: **ADMIN**, **ANALYST**, **VIEWER** (see `backend-06-security-architecture.md` / `tech-07-security-architecture.md` for the full matrix). Where the frontend shows a control to a role that will actually get a `403` from the backend, that mismatch is called out explicitly in **Error Conditions** -- it is a real, observable behavior, not a documentation error.
Input	What the user provides (typed text, a selection, a click) before the action runs.
Output	What the user sees as a direct result.
Backend Process	The exact `frontend/lib/api.ts` function and the HTTP method + path it calls (all paths are relative to `/api/v1` unless noted). Full request/response schemas live in `backend-02-api-reference.md`; this column exists so you don't have to cross-reference for the common case of "which endpoint does this button hit."
Database Effect	The Postgres table(s) written or read, by real table name. "None" means the action is entirely client-side (no network call at all) or read-only against already-loaded data.
Error Conditions	What can go wrong, and how the UI actually represents it (never invented -- traced to the real `.catch()` /error-state handling in the component).
Related Features	Cross-references to other functions in this chapter or other chapters.

A **How to Use** numbered list follows the table for any function with more than one meaningful step.

Two functions -- the global navigation bar and the pivot search box -- appear on nearly every authenticated page. They're documented once, in **Global / Shared Elements**, rather than repeated verbatim under every page section.

Global / Shared Elements

These render identically (same component, same code path) on most authenticated pages. Page-specific sections below note only where a page *differs* from what's documented here.

Global Navigation Bar

Field	Detail
Location	`components/dashboard/WorkspaceNav.tsx` -- rendered on `/lookup/new`, `/lookup/[id]`, `/dashboard`, `/dashboard/provider-health`, `/basket`, `/cases`, `/cases/[id]`, `/providers`, and `/admin`. **Not** rendered on `/`, `/login`, or `/register`.
Purpose	Lets an analyst move between the five nav groups -- **Command** (Dashboard), **Intelligence** (Basket), **Analysis** (Cases), **Operations** (Provider Health), and **Administration** (Providers + Administration) -- without hunting for a link on each page.
Role Required	None to see the bar itself. The **Administration** group renders as an expandable dropdown containing both "Providers" and "Administration" links only when `GET /auth/me` reports `role === "admin"`; every other logged-in role sees a single "Providers" pill in that slot with no dropdown and no "Administration" link at all. This is a UX convenience only -- see **Error Conditions**.
Input	Click on any nav pill, or (for the Administration group, admins only) click "Manage" to open the dropdown first.
Output	Client-side navigation (Next.js router) to the target route. The active group's caption and pill are highlighted using the *longest matching* route prefix, so visiting `/dashboard/provider-health` lights up "Operations," not "Command," even though both routes share the `/dashboard` prefix. Visiting any `/lookup/*` route lights up the **Intelligence** caption (but not the Basket pill itself), since Investigate has no dedicated nav link of its own.
Backend Process	`listBasket()` (`GET /basket`) to populate the numeric badge on the Basket pill; `getCurrentUser()` (`GET /auth/me`) to decide whether to show the Administration dropdown. Both run once per route change.
Database Effect	None (read-only).
Error Conditions	If the logged-in user's role is **ANALYST** or **VIEWER**, `listBasket()` (which requires `basket:manage`, granted only to ADMIN/ANALYST) fails for a VIEWER specifically -- the badge count is caught and set to `null`, so a VIEWER simply sees "Basket" with no number, not an error. The Administration dropdown check (`getCurrentUser`) never itself fails for a logged-in user; a non-admin simply never sees the dropdown, matching the fact that every `/api/v1/admin/*` and `/api/v1/runtime/*` route independently re-enforces `user:manage` / `provider:manage` server-side regardless of what this nav shows.
Related Features	Pivot Search Bar (below); every page section's "Role Required" field.

Pivot / Investigate Search Bar

Field	Detail
Location	`components/dashboard/TopSearchBar.tsx` -- rendered alongside the Global Navigation Bar on every page listed above. Distinct from the hero search box on the home page (`/`), which is its own inline form.
Purpose	Lets an analyst start a brand-new investigation from anywhere in the app without navigating back to the home page first.
Input	Free-text IOC value (IP, domain, URL, hash, CVE, threat actor, YARA rule, etc.).
Output	Navigates to `/lookup/new?value=<encoded value>`. If the new value is typed while already on `/lookup/new`, the page fully remounts (it keys its root component on the raw `?value=` query param) so the previous IOC's provider results, event log, and evidence highlights don't bleed into the new investigation's state.
Role Required	Must be logged in; if not, submitting redirects to `/login?next=/lookup/new?value=...` first.
Backend Process	None directly -- it only constructs a URL. The actual investigation call happens on `/lookup/new` itself (see **Start Investigation** below).
Database Effect	None.
Error Conditions	Submitting an empty string is a no-op (the submit button is disabled while the input is blank).
Related Features	Start Investigation (`/lookup/new`); Home Page Investigate box.

Session Handling (Token Refresh and Expiry)

Not a clickable function, but a real, observable background behavior worth documenting once rather than under every page: every authenticated API call in `lib/api.ts` goes through `authedFetch()`, which retries a request exactly once after a transparent token refresh if the first attempt returns `401`. If the refresh itself fails (the stored refresh token is missing, expired, or rejected), both tokens are cleared from `localStorage` and the user is redirected to `/login?next=<the page they were on>`. For the live investigation stream specifically (`streamLookup()`), the same 401-then-refresh-then-retry logic runs once before the SSE `POST` itself; if the retried request also fails, the stream reports `"Your session expired. Please sign in again."` rather than a generic network error, and the page redirects to `/login`.

Page: Sign In (`/login`)

Sign In

Field	Detail
Location	`/login`, `app/login/page.tsx`.
Purpose	Authenticate an existing account and obtain an access/refresh token pair.
Role Required	None (this is how you obtain a role in the first place).
Input	Email, password.
Output	On success, redirects to whatever `?next=` was set to (default `/`). On failure, shows "Invalid email or password." beneath the form -- the same message regardless of whether the email doesn't exist or the password is wrong, so the error never discloses which account exists.
Backend Process	`login(email, password)` → `POST /auth/login`.
Database Effect	Reads `users` (email/password-hash lookup); on success, updates that row's `last_login_at`.
Error Conditions	Any non-2xx response (wrong credentials, disabled account, malformed body) collapses to the single generic error message above. A disabled account (`is_active = false`) is rejected the same way as a wrong password -- there is no separate "your account has been disabled" message on this page.
Related Features	Register; Session Handling (above); Reset Password (Admin page) for how a disabled/locked-out account gets re-enabled.

How to Use:

1. Enter your email and password.
2. Click **Sign in**.
3. On success you land back wherever you were trying to go before being redirected here (e.g. a specific `/lookup/new?value=...` URL), or the home page if you arrived directly.

Page: Create Account (`/register`)

Create Account (Bootstrap Registration)

Field	Detail
Location	`/register`, `app/register/page.tsx`.
Purpose	Create a brand-new user account. The page's own subtitle states the real, current behavior plainly: *"Self-registration only works on a brand-new installation with no existing admin. If one already exists, ask an administrator to create your account from the Administration page."*
Role Required	None to attempt it -- but the backend only allows this to succeed **once**, for the very first account on the instance, which is automatically granted the **ADMIN** role. Every subsequent registration attempt is rejected regardless of who's asking.
Input	Full name (optional), email (required), password (required, minimum 8 characters, enforced client-side via `minLength`).
Output	On success, the new account is immediately logged in (the page calls `login()` right after `register()` succeeds) and redirected to `/`. On failure, the backend's own rejection detail is shown inline (e.g. an "admin already exists" style message once bootstrap has happened).
Backend Process	`register(email, password, fullName)` → `POST /auth/register`, followed immediately by `login(email, password)` → `POST /auth/login`.
Database Effect	Inserts one row into `users` (only while that table is empty) with `role = admin`.
Error Conditions	Once any account exists, every subsequent registration attempt fails server-side, and the failure's `detail` text is surfaced directly rather than a generic message. There is no path to self-register a second (or `analyst` / `viewer`) account -- that must be done from **Administration → Users → New User** (see below).
Related Features	Sign In; Administration → Create User (the only way to add accounts after bootstrap).

How to Use:

1. Fill in full name, email, and a password of at least 8 characters.
2. Click **Create account**.
3. If this is truly the first account ever created on this installation, you're logged in immediately as ADMIN. Otherwise, you'll see an error and need to ask an existing administrator to create your account instead.

Page: Home / Investigate (`/`)

Start Investigation (Home Search Box)

Field	Detail
Location	`/`, `app/page.tsx`.
Purpose	The platform's primary entry point: type any IOC and begin a live investigation.
Role Required	Must be logged in to actually run a lookup (`lookup:create`, granted to ADMIN and ANALYST only). Typing and submitting works for anyone, logged in or not -- see **Error Conditions**.
Input	Free-text IOC value, typed into the search box or pre-filled by clicking one of the four example buttons (`8.8.8.8`, `malicious-example.com`, `CVE-2024-3400`, `T1059`).
Output	Navigates to `/lookup/new?value=<encoded value>`, where the actual investigation begins (see that page's **Start Investigation (SSE Stream)** entry).
Backend Process	None on this page itself -- purely a client-side redirect. The real API call happens on the destination page.
Database Effect	None.
Error Conditions	If not logged in, submitting redirects to `/login?next=/lookup/new?value=...` instead of starting the investigation directly -- so an anonymous visitor never actually reaches `lookup:create`'s enforcement point; they're stopped by the frontend's own login gate first, one step earlier. A VIEWER *is* logged in and *will* reach `/lookup/new`, but the SSE stream itself will then fail server-side with a `403` from `lookup:create` -- see that page's error handling.
Related Features	Pivot Search Bar; Quick-Fill Example IOC; Start Investigation (`/lookup/new`).

Quick-Fill Example IOC

Field	Detail
Location	`/`, four buttons directly beneath the search box.
Purpose	Let a new user see a real, live investigation without having to think of an IOC to type -- these are genuine values that return real provider data, not a canned demo.
Role Required	None to click (it only fills the text box); the same login requirement as **Start Investigation** applies once submitted.
Input	Click one of `8.8.8.8`, `malicious-example.com`, `CVE-2024-3400`, `T1059`.
Output	The clicked value populates the search input; the user must still press **Investigate** (or Enter) to submit it.
Backend Process	None (client-side state update only).
Database Effect	None.
Error Conditions	None distinct from Start Investigation.
Related Features	Start Investigation (Home Search Box).

AI Quick Switch

Field	Detail
Location	`/`, `components/dashboard/AiQuickSwitch.tsx` -- shown only when logged in, directly beneath the search box.
Purpose	Choose which AI backend (Ollama, Claude/Anthropic, AWS Bedrock, Gemini, or Groq) will analyze the *next* investigation started anywhere in the app, with no restart.
Role Required	Effectively **ADMIN only**, even though the control itself is visible to every logged-in role. See **Error Conditions**.
Input	Select a backend from the dropdown.
Output	A colored dot next to the dropdown indicates whether the selected backend is configured (green) or not (gray). A "Manage" link jumps to `/providers` for full credential setup.
Backend Process	Populates from `listAIProviders()` (`GET /runtime/ai-providers`) and `getActiveAIBackend()` (`GET /runtime/ai-active`); selecting a new value calls `setActiveAIBackend(backend)` (`POST /runtime/ai-active`).
Database Effect	Reads/updates the `is_active` flag on the matching row in `provider_runtime_configs`; the change is also written to `config_audit_log`.
Error Conditions	`listAIProviders()` requires the `provider:manage` permission, which only **ADMIN** holds. For an ANALYST or VIEWER, that call fails, the component's `.catch()` swallows the error silently, `providers` stays an empty array, and **the entire control renders nothing at all** (`if (providers.length === 0) return null;`) -- a non-admin never even sees an "AI:" selector on the home page, despite the component being unconditionally rendered in the page's JSX for every logged-in user. Even in the hypothetical case where an admin's session role changed mid-session, `setActiveAIBackend` itself also requires `provider:manage` and would `403`.
Related Features	Manage Providers (`/providers`) AI Providers tab -- the full configuration surface this control is a shortcut into; Final Assessment Panel's "Generated by" badge, which shows which backend actually produced a given investigation's result.

Sign Out

Field	Detail
Location	`/`, top-right corner, visible whenever logged in.
Purpose	End the current session.
Role Required	Any logged-in role.
Input	Click.
Output	Both `access_token` and `refresh_token` are removed from `localStorage`; the page's own state flips to the logged-out view (search box still visible, but "Sign in"/"Register" replace "Sign out"). No navigation occurs -- you stay on `/`.
Backend Process	`logout()` -- purely local; there is no server-side call and no server-side session to invalidate (JWTs are stateless until their own expiry, or until an admin-triggered password reset bumps the account's `token_version`).
Database Effect	None.
Error Conditions	None -- this cannot fail.
Related Features	Sign In; Session Handling.

Open Administration Console (Home Page Shortcut)

Field	Detail
Location	`/`, top-right corner, visible only when `GET /auth/me` reports `role === "admin"`.
Purpose	The home page is where most users land immediately after signing in, and (unlike `/providers`, `/cases`, or `/basket`) it renders no Global Navigation Bar at all -- without this shortcut, an administrator would have no visible path to `/admin` from here short of typing the URL directly.
Role Required	ADMIN (both the button's visibility and the destination page's own guard).
Input	Click.
Output	Navigates to `/admin`.
Backend Process	`getCurrentUser()` (`GET /auth/me`) on page load, to decide whether to render the button at all.
Database Effect	None.
Error Conditions	None distinct from the Administration page's own guard (see that section).
Related Features	Administration (`/admin`), all functions.

Page: Live Investigation (`/lookup/new`)

This is the platform's core screen -- the composition root for a live, in-progress investigation. It opens a Server-Sent Events (SSE) connection and renders each panel below as its corresponding event arrives. There is no WebSocket anywhere in this product; the entire live-progress experience is SSE. For the full event lifecycle and provider-fan-out mechanics, see `backend-02-api-reference.md`'s `/lookup/stream` entry and `tech-02-dataflow.md`; this section focuses on what each resulting panel lets the user *do*.

Start Investigation (SSE Stream)

Field	Detail
Location	`/lookup/new?value=<IOC>`, `app/lookup/new/page.tsx`. Triggered automatically on page load/mount (not a button on this page itself -- the button that got you here was on `/`, the Pivot Search Bar, or another page's "Investigate" action).
Purpose	Detect the IOC's type, fan out to every enabled provider that supports that type concurrently, correlate the results, and generate an AI-backed final assessment -- all streamed back live rather than waiting for one big response at the end.
Role Required	**ADMIN or ANALYST** (`lookup:create`).
Input	The raw IOC value from the `?value=` query parameter; optionally (not exposed as UI on this page, but supported by the underlying `streamLookup()` client function) a specific subset of `provider_ids` or an `ai_backend` override for this one investigation.
Output	A sequence of SSE events consumed live: `detected` (IOC type identified) → N × `provider_result` → N × `provider_summary` → `correlation` → `final_assessment` → `done`. Each event updates exactly the panel(s) described below as it arrives; the header bar's verdict badge reads "pending" until `final_assessment` arrives, then flips to the real verdict, and a green "complete" badge appears once `done` fires.
Backend Process	`streamLookup(value, handlers, signal, options)` → `POST /lookup/stream` (the request itself, not a `GET`, despite being consumed as an event stream -- because `EventSource` can't send an `Authorization` header, this client uses `fetch()` with a manual `ReadableStream` reader instead).
Database Effect	Creates one row in `ioc_lookups`; as results arrive, inserts rows into `provider_results`, `ai_summaries`, and `correlation_edges`; on `final_assessment`, writes the primary result onto the `ioc_lookups` row itself (final verdict, risk/confidence scores) and inserts a row into `final_assessment_records`.
Error Conditions	An `error` SSE event (e.g. a mid-stream backend failure) is shown inline in a red banner with the server's message. If the access token is expired and the transparent refresh (see **Session Handling**) also fails, the stream reports "*Your session expired. Please sign in again.*" and redirects to `/login`. A **VIEWER** reaching this page at all (e.g. by pasting a `/lookup/new?value=...` URL directly) will have the initial `POST /lookup/stream` itself rejected with `403` before any event is ever emitted, since `lookup:create` is ADMIN/ANALYST only -- the page then shows that failure via the same red error banner (`Request failed with status 403`).
Related Features	Pivot Search Bar; Home Page Investigate box; every panel below, all of which are populated from this one stream.

View Threat Score Gauge

Field	Detail
Location	`components/dashboard/ThreatScoreGauge.tsx`, top of the main column.
Purpose	Give an at-a-glance 0-100 risk read the moment the final assessment arrives, without reading any prose.
Role Required	Same as the page (viewing only -- no separate permission).
Input	None (display only).
Output	A semicircular gauge showing `risk.overall_risk_score` (0-100), colored by a fixed threshold (≥75 red, ≥50 amber, ≥25 violet, below that green -- a display-only banding, distinct from the backend's own severity bands used for the Final Assessment's severity label), plus the plain-language severity label and the confidence score. Renders a pulsing skeleton with "Awaiting final assessment..." until the `final_assessment` event arrives.
Backend Process	Populated from the `final_assessment` SSE event's `risk` object -- no separate API call.
Database Effect	None (read-only display).
Error Conditions	None distinct from the stream itself failing (in which case the gauge simply never leaves its loading state).
Related Features	Final Assessment Panel's "Risk & Verdict" tab, which repeats the same numbers alongside malicious probability and analyst confidence; Verdict Analysis → Score Explanation, for *why* the number is what it is.

View Provider Progress Tracker

Field	Detail
Location	`components/dashboard/ProviderProgressTracker.tsx`, right-hand sidebar.
Purpose	Show, live, which providers have already answered and which are still running -- since providers are queried concurrently, they finish in whatever order they actually respond, not a fixed sequence.
Role Required	Same as the page.
Input	None (display only).
Output	A progress bar (`responded / totalExpected`) plus one row per provider that has already reported (name, category icon, and a status badge: OK / Error / Timeout / Rate Limited / Not Configured / Unsupported / No Data / Disabled), with anonymous "Provider N" placeholder rows (spinning icon) filling the gap up to the expected total, since the SSE stream only identifies a provider once it actually responds.
Backend Process	Derived entirely from accumulated `provider_result` SSE events; `totalExpected` is computed once from `getProviderHealth()` (`GET /providers/health`) filtered to providers whose `supported_types` include the detected IOC type.
Database Effect	None (read-only).
Error Conditions	If `getProviderHealth()` fails, `totalExpected` falls back to just counting whatever's arrived so far -- the progress bar becomes less accurate (it can briefly read 100% before every provider has truly finished) but never breaks.
Related Features	Provider Result Cards (below), which show the actual data behind each of these status badges; Provider Health page, the persistent (non-per-investigation) version of the same status vocabulary.

View Provider Result Cards / Expand Raw Data

Field	Detail
Location	`components/dashboard/ProviderCardGrid.tsx` / `ProviderCard.tsx`, main column.
Purpose	Show each provider's actual returned data and the AI's per-provider summary of it, side by side, one card per provider.
Role Required	Same as the page.
Input	Click a card's "Raw data" `<details>` disclosure to expand the full JSON payload that provider returned.
Output	Cards render OK-status providers first. Each card shows: a status badge; category and latency; a "cached" tag if the result was served from cache rather than a fresh call; a "Source" link to the provider's own page for that IOC, if one exists; every scalar/array field from that provider's data, formatted (the `internet_intelligence` provider's `osint_findings` array gets a dedicated attributed link list instead of the generic field renderer); and, once ready, an **AI Summary** section (threat level, reputation, confidence, interesting findings, relationships, unique observations) -- or, for a non-`ok` status, the plain statement *"No AI summary -- provider status is '<status>'."* rather than an empty or missing section.
Backend Process	Populated from `provider_result` and `provider_summary` SSE events -- no separate API call.
Database Effect	None (read-only display of already-streamed data).
Error Conditions	A provider that errored still gets a card (with its `error_message` shown), never a silently missing one -- a failed or skipped provider is exactly as visible as a successful one, just distinctly labeled.
Related Features	Provider Progress Tracker; Provider Health page (the same status vocabulary, aggregated over time instead of per-investigation).

View Final Assessment (Tabs)

Field	Detail
Location	`components/dashboard/FinalAssessmentPanel.tsx`, main column.
Purpose	The single consolidated AI-authored (or deterministic-fallback) conclusion for the investigation, organized into five tabs: Executive Summary, Technical Summary, Threat Assessment, Relationships, and Risk & Verdict.
Role Required	Same as the page.
Input	Click a tab to switch views.
Output	**Executive Summary** -- plain-language summary plus the final verdict badge and rationale. **Technical Summary** -- the technical write-up. **Threat Assessment** -- narrative plus agreeing/disagreeing provider pills and a numbered supporting-evidence list. **Relationships** -- prose summary of what this IOC connects to. **Risk & Verdict** -- the same `overall_risk_score` / `confidence_score` / `severity` / `reputation` / `malicious_probability` / `analyst_confidence` fields as the gauge, in tile form. The header also shows which AI backend/model actually produced this result (e.g. "Generated by Claude · claude-sonnet-...") or, if no AI call happened at all because there was no provider evidence to assess, "No AI call (no evidence to assess)" -- the platform never fabricates an AI attribution.
Backend Process	Populated entirely from the `final_assessment` SSE event -- no separate API call.
Database Effect	None (read-only display; the underlying write happened when the stream produced this event -- see **Start Investigation**).
Error Conditions	Until `final_assessment` arrives, the whole panel renders a pulsing "Final assessment pending -- waiting for all providers to finish" placeholder rather than an empty or broken layout.
Related Features	Threat Score Gauge; AI Comparison Panel (run the *same* evidence through a different backend); Verdict Analysis panel (interrogate this conclusion in detail); Export (every export format is built from this exact object).

View Relationship Graph / Toggle List View / Click Node to Pivot

Field	Detail
Location	`components/dashboard/RelationshipGraph.tsx`, main column.
Purpose	Visualize what this IOC connects to (shared infrastructure, related files, campaigns, etc.) as a force-directed graph, with a fully accessible table fallback.
Role Required	Same as the page.
Input	Click **"View as list"** / **"View as graph"** to toggle rendering mode; click any node in graph mode; hover any node or edge for a tooltip.
Output	**Graph mode**: a canvas-rendered force-directed graph (`react-force-graph-2d`, client-only), nodes colored by IOC-type family (network/infrastructure, file/hash, threat-actor/campaign, vulnerability/technique, host/endpoint, or a neutral fallback), edges rendered more translucent the lower their `confidence`. Clicking a node navigates to `/lookup/new?value=<that node's value>`, starting a brand-new investigation. **List mode**: the identical data as a plain HTML table (Source / Relationship / Target / Confidence / Source(s)) -- added specifically because the canvas graph itself is not keyboard- or screen-reader-accessible.
Backend Process	Populated from the `correlation` SSE event -- no separate API call for display. Clicking a node re-triggers **Start Investigation (SSE Stream)** for the new value.
Database Effect	None from viewing; clicking a node creates a brand-new `ioc_lookups` row via the resulting investigation, same as any fresh Start Investigation.
Error Conditions	If no relationships were discovered, both modes are replaced by a single "No relationships discovered yet" empty state (the list-view toggle button itself is hidden in this case, since there's nothing to view either way).
Related Features	Pivot panel (below) -- a curated, ranked *list* of the same underlying relationships, versus this graph's *visual* map of all of them; MITRE ATT&CK Matrix.

View MITRE ATT&CK Matrix / Click Technique

Field	Detail
Location	`components/dashboard/MitreMatrix.tsx`, main column.
Purpose	Show which MITRE ATT&CK tactics/techniques this investigation's evidence actually supports, grouped by tactic column.
Role Required	Same as the page.
Input	Hover a technique cell for its AI-authored rationale (tooltip); click a cell to open that technique's official page on `attack.mitre.org` in a new tab.
Output	One column per tactic, one cell per technique, each showing the technique ID, name, and kill-chain stage. Techniques the AI inferred from context rather than one a provider directly surfaced are marked with an **"AI-inferred"** badge -- the matrix never presents an inferred technique as if a provider had reported it directly.
Backend Process	Populated from `final_assessment.mitre_mappings` (part of the same `final_assessment` SSE event) -- no separate API call.
Database Effect	None (read-only).
Error Conditions	"No MITRE ATT&CK techniques identified" is shown, correctly, when the evidence doesn't support mapping to any known technique (e.g. a genuinely benign IOC like Google's public DNS resolver) -- this is treated as a valid, expected outcome, not an error state.
Related Features	Detection Rules panel (techniques often drive which rule format makes sense); Final Assessment's Threat Assessment tab.

View Detection Rules (from Final Assessment)

Field	Detail
Location	`components/dashboard/DetectionRulesPanel.tsx`, main column. This is distinct from the Threat Hunting Center's own "Create Detection" action (below) -- this panel shows whatever detection logic the **Final Assessment step itself** already generated as part of the investigation, with no extra click required.
Purpose	Surface ready-to-use detection logic (Sigma, YARA, Splunk SPL, Sentinel KQL, Elastic, QRadar AQL, Suricata, Snort, Zeek, etc.) the AI produced as part of the same final assessment, when the evidence supports writing one.
Role Required	Same as the page (viewing only).
Input	Click a format tab; click **Copy** on any rule.
Output	One tab per format actually present, each showing the rule's title and full text in a code block; **Copy** copies that rule's raw text to the clipboard with a 1.5-second "Copied" confirmation.
Backend Process	Populated from `final_assessment.detection_rules` -- no separate API call. Clipboard writes are local-only (`navigator.clipboard.writeText`).
Database Effect	None.
Error Conditions	"No detection logic generated for this IOC type" is the correct, expected message when there's no malicious behavior to write a rule against.
Related Features	Threat Hunting Center's **Create Detection** action (a separate, on-demand generation you control the format for); Threat Hunting Center's **Hunt This IOC** action.

View Recommended Actions / Investigation Priorities / Incident Response

Field	Detail
Location	`components/dashboard/RecommendedActionsPanel.tsx`, main column -- three cards rendered side by side.
Purpose	Surface the AI's suggested next steps, grounded in what was actually found, split into three categories: general recommended actions, what to prioritize investigating next, and incident-response-specific recommendations.
Role Required	Same as the page (viewing only).
Input	None (display only).
Output	Three bulleted lists (or "None identified." per card if empty). Before the final assessment arrives, all three cards render with their titles visible and a pulsing skeleton body -- deliberately, so an absent section reads as "still loading," never as "this feature doesn't exist."
Backend Process	Populated from `final_assessment.recommended_actions` / `.investigation_priorities` / `.incident_response_recommendations` -- no separate API call.
Database Effect	None.
Error Conditions	None beyond the loading-skeleton behavior above.
Related Features	Pivot panel's own "What should I do next?" (a separate, on-demand AI call producing a similarly-shaped but independently generated list, tied to specific pivot targets).

AI Comparison: Select Backend and Analyze

Field	Detail
Location	`components/dashboard/AiComparisonPanel.tsx` -- rendered on `/lookup/new` **only**, once the investigation is complete (`isDone && lookupId`). **Not rendered on** `/lookup/[id]` at all -- re-analyzing a previously completed investigation with a different backend is only offered from the live view.
Purpose	Re-run just the final-assessment step against a different AI backend (e.g. compare Claude vs. Groq vs. local Ollama), using the exact same already-collected evidence -- no provider is re-queried.
Role Required	**ADMIN or ANALYST** -- this calls the same `lookup:create` permission as starting a fresh investigation, not `analysis:generate` .
Input	Select a configured AI backend from the dropdown (backends already run against this lookup are marked "(already run)" but remain selectable -- re-running the same backend is allowed).
Output	A new comparison card appears (or updates), labeled "Original" (the investigation's primary result) or "Comparison," showing that backend/model's executive summary, verdict badge, risk score, confidence score, and malicious probability. Every prior comparison remains listed -- results are durable across a page refresh, not just kept in memory for the tab's lifetime.
Backend Process	`listAIProviders()` (`GET /runtime/ai-providers`) to populate the dropdown; `reanalyzeLookup(lookupId, backend)` (`POST /lookup/{id}/reanalyze`) to run the comparison; `listAssessments(lookupId)` (`GET /lookup/{id}/assessments`) to (re)fetch every stored result after each run.
Database Effect	Inserts a new row into `final_assessment_records` (with `is_primary = false` , `ai_backend` / `ai_model` set to the chosen backend). The original investigation's primary assessment on `ioc_lookups` is never overwritten.
Error Conditions	If re-analysis fails (e.g. the selected backend's credentials are invalid, or the backend is unreachable), the error message from the backend is shown inline beneath the selector; no partial/broken comparison card is added.
Related Features	Final Assessment Panel; AI Quick Switch (chooses the backend for the *next new* investigation, not a re-analysis of an existing one).

Verdict Analysis: WHY?

Field	Detail
Location	`components/dashboard/VerdictAnalysisPanel.tsx`, main column, first of six modes. Rendered on both `/lookup/new` (post-completion) and `/lookup/[id]`.
Purpose	Ask the AI to justify the final verdict, citing the specific evidence behind each reason -- so the verdict is never just "trust the score."
Role Required	**ADMIN or ANALYST** (`analysis:generate`).
Input	Click the "WHY?" button.
Output	The verdict restated in one line, followed by a numbered list of reasons, each with a **Show Receipts** link; and, if applicable, a caveat. Cached after first generation -- clicking again while the same mode is already active does not re-call the API.
Backend Process	`explainWhyMalicious(lookupId)` → `POST /lookup/{id}/analysis/why`.
Database Effect	None written (a read-only AI generation over already-persisted evidence); the AI call itself is logged as part of the platform's normal AI-usage tracking, not a separate table specific to this button.
Error Conditions	Generation failure shows the backend's error message inline; each reason with zero cited evidence still renders (with "No specific reasons could be grounded in the evidence" if the whole list is empty) rather than a silent gap. A **VIEWER** clicking this button gets a `403` from `analysis:generate`, surfaced as the caught error message inline in the panel -- the button itself is not hidden from a VIEWER.
Related Features	Show Receipts (below); Evidence Ledger; the other five Verdict Analysis modes.

Verdict Analysis: What Is This?

Field	Detail
Location	Same panel, second mode.
Purpose	A plain-language explanation of what the IOC actually is, alongside a more technical explanation, for an analyst who needs to explain the finding to someone less technical (or just wants the two framings side by side).
Role Required	ADMIN or ANALYST (`analysis:generate`).
Input	Click "What is this?".
Output	Plain-language summary; technical explanation; a confidence narrative; related infrastructure pills (if any); Show Receipts link.
Backend Process	`explainWhatIsThis(lookupId)` → `POST /lookup/{id}/analysis/what-is-this`.
Database Effect	None written.
Error Conditions	Same pattern as **WHY?** above (inline error message; VIEWER gets 403).
Related Features	WHY?; Evidence Ledger.

Verdict Analysis: Score Explanation

Field	Detail
Location	Same panel, third mode.
Purpose	Break down *how* the numeric risk score was arrived at, component by component, each tied to specific evidence -- the AI narrates the platform's own deterministic scoring engine's output rather than inventing a new explanation.
Role Required	ADMIN or ANALYST (`analysis:generate`).
Input	Click "Score Explanation".
Output	One block per scoring component (its name, its contribution, and a Show Receipts link), followed by a plain-language summary sentence.
Backend Process	`explainScore(lookupId)` → `POST /lookup/{id}/analysis/score-explanation` .
Database Effect	None written.
Error Conditions	Same pattern as WHY? above.
Related Features	Threat Score Gauge; Final Assessment's Risk & Verdict tab -- this mode explains the exact numbers those two panels display, which were computed by the deterministic scoring engine *before* any AI call and are never altered by the AI afterward.

Verdict Analysis: Intelligence Conflicts (Disagreement)

Field	Detail
Location	Same panel, fourth mode.
Purpose	Specifically surface where providers disagree with each other -- the single most important thing to read before trusting an aggregate score, since one dissenting or erroring provider can pull a score in a direction the fuller picture doesn't support.
Role Required	ADMIN or ANALYST (`analysis:generate`).
Input	Click "Intelligence Conflicts".
Output	Four labeled sections -- Agreement, Conflict, Missing Data, and Most Reliable Evidence -- plus a Show Receipts link.
Backend Process	`explainDisagreement(lookupId)` → `POST /lookup/{id}/analysis/disagreement` .
Database Effect	None written.
Error Conditions	Same pattern as WHY? above.
Related Features	Final Assessment's Threat Assessment tab (agreeing/disagreeing provider pills); False Positive Check; Challenge This Verdict.

Verdict Analysis: False Positive Check

Field	Detail
Location	Same panel, fifth mode.
Purpose	Ask the AI to specifically assess how likely it is that a flagged verdict is wrong, and why (categorized: e.g. shared infrastructure, sinkhole, research/scanner activity, etc.).
Role Required	ADMIN or ANALYST (`analysis:generate`).
Input	Click "False Positive Check".
Output	A badge stating either "Possible false positive" or "No false-positive indicators found," candidate category pills (if any), a written explanation, and a Show Receipts link.
Backend Process	`checkFalsePositive(lookupId)` → `POST /lookup/{id}/analysis/false-positive` .
Database Effect	None written.
Error Conditions	Same pattern as WHY? above.
Related Features	Challenge This Verdict (a more adversarial version of the same instinct); Intelligence Conflicts.

Verdict Analysis: Challenge This Verdict

Field	Detail
Location	Same panel, sixth mode.
Purpose	A deliberate red-team tool: instead of defending its own conclusion, the AI is asked to argue *against* it -- listing supporting evidence, contradictory evidence, missing evidence, and an alternative explanation, then stating its own final confidence in the original verdict after having tried to break it.
Role Required	ADMIN or ANALYST (`analysis:generate`).
Input	Click "Challenge This Verdict".
Output	Supporting-evidence list (with Show Receipts per item), contradictory-evidence list (same), a missing-evidence list, an alternative-explanation paragraph, and a final-confidence badge (low/medium/high) with its own rationale.
Backend Process	`challengeVerdict(lookupId)` → `POST /lookup/{id}/analysis/challenge` .
Database Effect	None written.
Error Conditions	Same pattern as WHY? above.
Related Features	False Positive Check; Intelligence Conflicts; Evidence Ledger.

Show Receipts (Shared Control)

Field	Detail
Location	`components/dashboard/ShowReceiptsLink.tsx` -- appears next to nearly every AI-generated claim across the Verdict Analysis panel and Investigation Copilot, wherever that claim cites `evidence_ids`.
Purpose	Let an analyst verify any individual AI claim against its actual source record, rather than taking the claim on faith.
Role Required	Same as whatever panel it appears in (viewing only -- no separate permission of its own).
Input	Click "Show receipts (N)".
Output	Scrolls the page to the Evidence Ledger and highlights exactly the cited rows (a colored border), filtering the ledger down to just those items until cleared. If a claim cites zero evidence, the control instead renders the plain italic text "No supporting evidence cited" -- never a clickable link to nothing.
Backend Process	None -- purely a client-side scroll/highlight against the already-loaded Evidence Ledger.
Database Effect	None.
Error Conditions	None.
Related Features	Evidence Ledger (below); every Verdict Analysis mode; Investigation Copilot.

View / Filter Evidence Ledger

Field	Detail
Location	`components/dashboard/EvidencePanel.tsx`, main column, shown once the investigation is complete.
Purpose	The deterministic (never AI-generated) record every AI claim elsewhere on the page cites back to -- built directly from real provider summaries and correlation edges, so any claim can be checked against a traceable source.
Role Required	**ADMIN, ANALYST, or VIEWER** (`evidence:read` -- the one Verdict-Analysis-adjacent permission granted to all three roles, since it's read-only).
Input	Click a filter pill (evidence type, e.g. Detection / Reputation / Relationship / Malware association / MITRE technique) to narrow the list; click a row to expand its full detail.
Output	One row per evidence item: type badge, claim text, and a confidence percentage (colored red ≥75%, amber ≥40%, muted below). Expanded, a row shows its source label, related IOC (if any), observed/recorded timestamps, interpretation text, and a link to the original source.
Backend Process	`getEvidence(lookupId)` → `GET /lookup/{id}/analysis/evidence`.
Database Effect	Reads `evidence_items` (populated during the original investigation run, not by viewing this panel).
Error Conditions	"No evidence recorded for this lookup" if the list is genuinely empty; a fetch failure shows the caught error message inline.
Related Features	Show Receipts; every Verdict Analysis mode; Investigation Copilot.

View Recommended Pivots / Investigate a Pivot

Field	Detail
Location	`components/dashboard/PivotPanel.tsx`, main column, first sub-section.
Purpose	A deterministic, ranked list of every IOC directly related to the seed indicator (never AI-generated -- a pure sort over the correlation graph), so an analyst can jump straight to the next most relevant thing to check.
Role Required	**ADMIN, ANALYST, or VIEWER** to view the list (`lookup:read`); investigating a suggested pivot requires `lookup:create` (ADMIN/ANALYST) the same as any new investigation.
Input	Click any pivot row.
Output	Each row shows the related IOC's value, type, relationship label, number of corroborating providers, and a confidence percentage, plus a relevance badge (high/medium/low). Clicking navigates to `/lookup/new?value=<that IOC>`.
Backend Process	`getPivots(lookupId)` → `GET /lookup/{id}/pivots`.
Database Effect	Read-only against `correlation_edges`; clicking through creates a new `ioc_lookups` row via the resulting fresh investigation.
Error Conditions	"No directly related indicators discovered yet" if the list is empty. A VIEWER can see and read this list but will get a `403` on the resulting investigation if they click through (same as any Start Investigation attempt).
Related Features	Relationship Graph (the visual, full version of the same underlying data); Ask AI for Next Actions (below).

Ask AI for Next Actions

Field	Detail
Location	Same panel, second sub-section ("What should I do next?").
Purpose	An AI-generated, prioritized list of concrete next steps, distinct from the deterministic pivot list above -- this one reasons about *why* a given next step matters, not just *what* else is related.
Role Required	**ADMIN or ANALYST** (`analysis:generate`).
Input	Click "Ask AI" (relabels to "Regenerate" after the first run).
Output	One card per suggested action: the action itself, a priority badge (low/medium/high), a rationale, and -- if the action targets a specific IOC -- an "Investigate <value>" button that jumps straight to a fresh investigation of it.
Backend Process	`getNextActions(lookupId)` → `POST /lookup/{id}/analysis/next-actions`.
Database Effect	None written.
Error Conditions	Failure shows an inline error; "No high-value next actions identified" is a valid, expected empty result. A VIEWER clicking "Ask AI" here gets a `403`.
Related Features	Recommended Pivots; Recommended Actions panel (the automatically-generated version from the original final assessment, versus this on-demand regeneration).

Ask AI for Intelligence Gaps

Field	Detail
Location	Same panel, third sub-section ("What don't we know?").
Purpose	Have the AI identify what's *missing* from the investigation -- data that would strengthen or change the conclusion if it existed -- and how to go get it.
Role Required	**ADMIN or ANALYST** (`analysis:generate`).
Input	Click "Ask AI" (relabels to "Regenerate").
Output	One card per identified gap: the gap itself, and "How to close it."
Backend Process	`getIntelligenceGaps(lookupId)` → `POST /lookup/{id}/analysis/gaps` .
Database Effect	None written.
Error Conditions	Failure shows an inline error; "No significant intelligence gaps identified" is a valid empty result. VIEWER gets `403` .
Related Features	Ask AI for Next Actions; Evidence Ledger.

Hunt This IOC (Threat Hunting Center)

Field	Detail
Location	`components/dashboard/HuntingCenterPanel.tsx` , main column.
Purpose	Generate copyable hunting queries, across nine SIEM/EDR/network query languages (Sigma, Splunk SPL, Sentinel KQL, Elastic, QRadar AQL, Chronicle YARA-L, Suricata, Snort, Zeek), that an analyst can run in their own environment -- both an exact-match query for the seed IOC itself, and expansion queries for related indicators actually present in this investigation's correlation graph (never invented).
Role Required	**ADMIN or ANALYST** (`hunting:generate`).
Input	Click "Hunt This IOC" (relabels to "Regenerate" after first run).
Output	A tab per query format with the exact-match query, a one-line description of what it detects, and a Copy button; below that, a list of hunting-expansion targets (related indicators with a rationale) and their own broader queries.
Backend Process	`huntThisIOC(lookupId, formats)` → `POST /lookup/{id}/hunt?formats=...` (repeated `formats` query params, one per requested format).
Database Effect	None written.
Error Conditions	Generation failure shows an inline error message. A VIEWER clicking this gets `403` .
Related Features	Detection Rules panel (the Final Assessment's own auto-generated rules); Create Detection (below, an on-demand equivalent scoped to one format at a time).

Copy Hunting Query

Field	Detail
Location	Same panel, every generated query.
Purpose	Get a generated query onto the clipboard for pasting directly into a SIEM/EDR console.
Role Required	Same as viewing the panel -- no separate permission (copying doesn't call the backend).
Input	Click "Copy" next to any query.
Output	The query text is written to the clipboard; the button shows a checkmark and "Copied" for 1.5 seconds.
Backend Process	None (`navigator.clipboard.writeText` , local only).
Database Effect	None.
Error Conditions	If clipboard access is denied by the browser, the copy silently fails (no error shown) -- the query text remains visible and selectable manually as a fallback.
Related Features	Hunt This IOC; Create Detection; Export (Markdown/JSON also copy-free but download instead of clipboard).

Create Detection

Field	Detail
Location	Same panel, bottom sub-section.
Purpose	Draft one full, production-style detection rule, on demand, in a chosen format -- with objective, data source, logic explanation, false-positive considerations, severity, and any MITRE technique IDs it maps to, not just the raw rule text.
Role Required	**ADMIN or ANALYST** (`hunting:generate`).
Input	Select a format from the dropdown (same nine formats as Hunt This IOC, plus YARA); click "Generate".
Output	A detection card: title, Copy-able rule text, objective, data source, logic explanation, false-positive considerations, a severity pill, and MITRE technique pills (if any).
Backend Process	`createDetectionRule(lookupId, format)` → `POST /lookup/{id}/detection?format=<format>` .
Database Effect	None written.
Error Conditions	Generation failure shows an inline error. VIEWER gets `403` .
Related Features	Hunt This IOC; Detection Rules panel.

Investigation Copilot: Ask a Question

Field	Detail
Location	`components/dashboard/InvestigationCopilot.tsx`, main column, shown once the investigation is complete.
Purpose	A persistent Q&A chat scoped entirely to this one investigation's evidence, correlation graph, and every note typed in this session -- so an analyst never has to re-paste context, and every answer stays grounded in what this lookup actually knows rather than general AI knowledge.
Role Required	**ADMIN or ANALYST** (`copilot:query`).
Input	Type a question and press Enter/click Send, or click a suggested starter ("Why is this suspicious?", "What changed since last week?", "Which provider has the strongest evidence?", "Is this likely a false positive?", "What should I investigate next?"), or click a suggested follow-up chip beneath a previous answer.
Output	Each turn shows the question, the answer, a Show Receipts link, and any AI-suggested follow-up questions as clickable chips. The whole conversation (as `Q:` / `A:` pairs) is sent back as context on every subsequent question, so the Copilot doesn't repeat itself or lose the thread.
Backend Process	`askCopilot(lookupId, question, notes)` → `POST /lookup/{id}/analysis/copilot`.
Database Effect	None written (evidence and correlation data are read-only inputs to this call; the conversation itself lives only in this browser tab's state and is lost on refresh -- there is no persisted chat history).
Error Conditions	A failed request shows an inline error beneath the chat; the input stays disabled while a request is in flight. VIEWER gets `403`.
Related Features	Show Receipts; Evidence Ledger; Verdict Analysis panel (the fixed-question equivalent of this free-form chat).

Add to Basket (Investigation Actions)

Field	Detail
Location	`components/dashboard/InvestigationActions.tsx`, right-hand sidebar.
Purpose	Add the current investigation's IOC to the analyst's personal Basket scratch-list without leaving the page.
Role Required	**ADMIN or ANALYST** (`basket:manage`).
Input	Click "Add to Basket".
Output	A confirmation message ("Added to basket.") for 3 seconds.
Backend Process	`addToBasket(iocValue)` → `POST /basket`.
Database Effect	Inserts one row into `basket_items`.
Error Conditions	Failure shows the caught error message in the same spot. A VIEWER clicking this gets `403` (`basket:manage` excludes VIEWER entirely).
Related Features	IOC Basket page (`/basket`), where this item then appears.

Add to Case (Investigation Actions)

Field	Detail
Location	Same component, second button.
Purpose	Link the current investigation's IOC directly to an existing case, so it's tracked as part of that incident rather than a disconnected search.
Role Required	**ADMIN or ANALYST** for both listing cases (`case:read` , actually granted to all three roles) and adding to one (`case:write` , ADMIN/ANALYST only).
Input	Click "Add to Case" to open the case picker, then click a case in the list.
Output	The picker lists every case by title ("No cases yet -- create one first." if none exist); clicking one links the IOC and shows "Added to case."
Backend Process	`listCases()` → `GET /cases` (to populate the picker, lazily on first open); `addCaseIOC(caseId, iocValue, iocType, lookupId)` → `POST /cases/{id}/iocs` .
Database Effect	Inserts one row into `case_iocs` , referencing the case, the IOC value/type, and (if available) the originating `lookup_id` .
Error Conditions	A VIEWER can open the picker and see the case list (`case:read` is granted to VIEWER) but clicking a case to add the IOC fails with `403` (`case:write` is ADMIN/ANALYST only) -- shown inline as "Failed to add to case."
Related Features	Case Detail page (`/cases/[id]`), where the IOC then appears; IOC Basket's own, informal equivalent.

Ask AI (Gemini Second Opinion)

Field	Detail
Location	`components/dashboard/AskAiPanel.tsx` , right-hand sidebar.
Purpose	Build a complete, escalation-focused analyst prompt from every piece of data pulled for this IOC (provider results, summaries, correlation), copy it to the clipboard, and open Google Gemini in a small docked popup window so an analyst can get a second opinion from their *own* Gemini account -- a deliberately different, independent AI perspective alongside the platform's own configured backend.
Role Required	Same as the page (viewing/using -- no backend permission of its own, since this makes no call to the HORIZON GRID backend at all).
Input	Click "Copy prompt & open Gemini box" (disabled until at least one provider has reported `ok`).
Output	The generated prompt is copied to the clipboard; a small (420×640) popup window opens to `gemini.google.com/app` , positioned next to the browser window. A message confirms the copy succeeded, or asks the user to copy manually if clipboard access was denied.
Backend Process	None against the HORIZON GRID backend -- `buildAiAnalysisPrompt()` (`lib/aiPrompt.ts`) runs entirely client-side against already-loaded provider data.
Database Effect	None.
Error Conditions	Google blocks `gemini.google.com` from loading inside an `<iframe>` (it sends `X-Frame-Options` /CSP headers, same as any other logged-in Google product), which is why this opens a real popup *window* instead of an embedded box -- documented in the component's own source comment as a deliberate, permanent workaround, not a bug. If the popup is already open, clicking again just re-focuses it rather than opening a duplicate.
Related Features	Investigation Copilot (the platform's own, evidence-grounded equivalent using its configured backend instead of an external one).

Export PDF

Field	Detail
Location	`components/dashboard/ExportMenu.tsx`, right-hand sidebar, shown once `lookupId` exists.
Purpose	A formatted, printable PDF version of the full assessment, rendered server-side.
Role Required	**ADMIN or ANALYST** (`lookup:export`).
Input	Click "Export PDF".
Output	Downloads `ioc-assessment-<lookupId>.pdf`.
Backend Process	Direct `fetch()` (not through `lib/api.ts`, but through the same `getApiUrl()` base-URL resolution) → `POST /lookup/{id}/export?format=pdf`.
Database Effect	Read-only against the already-persisted lookup/assessment data; no new rows written.
Error Conditions	A `404` (format not yet available on an older backend build) is shown as "Export format not yet available." rather than a raw error; other non-`ok` statuses show "Export failed with status <code>." A **VIEWER** clicking this button (which is not hidden from a VIEWER anywhere in the UI) gets a `403` from `lookup:export`, which surfaces as "Export failed with status 403." -- the button itself has no client-side role gate at all. Both PDF and CSV exports carry real, tested formula-injection/markup-injection protections against a maliciously-crafted IOC value or provider field ending up interpreted as executable spreadsheet/document content.
Related Features	Export Markdown, Export CSV, Export JSON (below) -- all four are offered side by side with no visual distinction between the two server-rendered formats and the two client-rendered ones.

Export Markdown

Field	Detail
Location	Same component, second button.
Purpose	A plain-text Markdown version of the assessment, suitable for pasting into a ticket, wiki page, or report.
Role Required	**None beyond being able to view the page at all** -- this makes no backend call. See **Error Conditions**.
Input	Click "Export Markdown".
Output	Downloads `ioc-assessment-<lookupId>.md`, built client-side from the already-loaded Final Assessment object (executive/technical summaries, threat assessment, verdict rationale, supporting evidence, MITRE mappings, recommended actions, investigation priorities, incident-response recommendations, and any detection rules).
Backend Process	**None.** `buildMarkdown()` runs entirely in the browser against data already streamed in; no HTTP request is made at all.
Database Effect	None.
Error Conditions	If no assessment has arrived yet, clicking shows "No assessment data available yet." rather than downloading an empty/broken file. **Because this button makes no server call, it is not gated by `lookup:export` (or any permission) at all** -- a VIEWER who can view a completed investigation (`lookup:read`) can successfully export it to Markdown even though the same VIEWER's PDF/CSV export attempts would `403`. This is a real, observable asymmetry between the two server-rendered and two client-rendered export formats.
Related Features	Export JSON (the same client-side-only exemption applies); Export PDF/CSV (the two permission-gated formats).

Export CSV

Field	Detail
Location	Same component, third button.
Purpose	A CSV version of the assessment, for spreadsheet import or bulk reporting workflows.
Role Required	**ADMIN or ANALYST** (`lookup:export`) -- same as PDF.
Input	Click "Export CSV".
Output	Downloads `ioc-assessment-<lookupId>.csv`.
Backend Process	`POST /lookup/{id}/export?format=csv`.
Database Effect	Read-only.
Error Conditions	Same as Export PDF (404 → "not yet available"; other failures → status-coded message; VIEWER → 403). Carries the same formula-injection protection as PDF (a field value that looks like a spreadsheet formula, e.g. starting with `=`, is neutralized before being written into the file).
Related Features	Export PDF.

Export JSON

Field	Detail
Location	Same component, fourth button.
Purpose	The raw Final Assessment object, exactly as the platform produced it, for programmatic use (feeding into another tool, archiving, etc.).
Role Required	**None beyond viewing the page** -- client-side only, same exemption as Export Markdown.
Input	Click "Export JSON".
Output	Downloads `ioc-assessment-<lookupId>.json` -- `JSON.stringify(assessment, null, 2)` of the exact object already rendered on the page.
Backend Process	**None** -- no HTTP request.
Database Effect	None.
Error Conditions	Same "No assessment data available yet." guard as Export Markdown if nothing has loaded. Same permission asymmetry as Export Markdown: works for a VIEWER even though `lookup:export` would otherwise deny them.
Related Features	Export Markdown.

Security Assessment: Select Tools and Profile

Field	Detail
Location	`components/dashboard/SecurityAssessmentPanel.tsx`, main column, shown once the investigation is complete, for IP/domain/URL/hostname-type IOCs.
Purpose	Configure an **active** check against the investigation's own target -- Nmap port/service scan, DNS enumeration, TLS certificate inspection, or an HTTP security-header check -- as opposed to every provider elsewhere on the page, which is purely passive (asking a third party what it already knows).
Role Required	The run-configuration form itself is shown only to **ADMIN or ANALYST** (checked client-side via `getCurrentUser()`, matching the real server-side `security_assessment:create` gate); **VIEWER** can see the read-only tool-health/results section below but never sees the configuration form at all.
Input	Click one or more tool buttons (disabled and marked "(unavailable)" for any tool the host doesn't currently support); select a profile from the dropdown, which is restricted to only the profile IDs valid across *every* currently-selected tool (in practice, "Standard" is the only option once more than one tool is selected, since only Nmap defines quick/standard/web -- every other tool defines only "standard").
Output	The form's own state -- no API call yet (that's the next function).
Backend Process	`getSecurityAssessmentProfiles()` (`GET /security-assessment/profiles`) and `getSecurityAssessmentToolHealth()` (`GET /security-assessment/tool-health`), both requiring `security_assessment:read` (all three roles).
Database Effect	None (read-only).
Error Conditions	"No security-assessment tool supports this IOC type" if none of the available tools apply.
Related Features	Run Security Assessment (below); Provider Health (a similar "is this working" question, but for passive providers instead of active tools).

Security Assessment: Run Assessment

Field	Detail
Location	Same panel, immediately below tool/profile selection.
Purpose	Actually execute the selected active check(s) against the investigation's own target, gated by two mandatory, explicit confirmations so nothing is ever scanned by accident or without authorization.
Role Required	**ADMIN or ANALYST** (`security_assessment:create` , enforced server-side regardless of what the frontend shows).
Input	Retype the exact target value into the confirmation box (must match the investigation's IOC value character-for-character); check "I am authorized to run active security checks against this target"; click "Run Security Assessment" (disabled until both conditions plus a tool/profile selection are satisfied).
Output	The run appears in the results list below with status "pending"/"running", polled every 2 seconds while any run is in flight, until it flips to "completed" or "failed".
Backend Process	`runSecurityAssessment(lookupId, toolIds, profile, targetConfirmation, authorizationConfirmed)` → `POST /security-assessment/{lookup_id}/run` ; polling via `listSecurityAssessmentRuns(lookupId)` → `GET /security-assessment/{lookup_id}/runs` .
Database Effect	Inserts one row into `security_assessment_runs` ; on completion, inserts one row per finding into `security_assessment_findings` . A completed run's findings feed back into the same investigation's risk score/verdict at the top of the page (a Security Assessment finding acts as a floor on `overall_risk_score` / `confidence_score` only -- never on `malicious_probability` , since a vulnerability finding is a different claim from "confirmed malicious actor").
Error Conditions	The submit button is disabled entirely until target confirmation exactly matches and authorization is checked -- both are enforced again server-side regardless. A run that fails outright shows its `error_message` inline in that run's row.
Related Features	Security Assessment: View Findings (below); the risk-score floor behavior described in `backend-09-security-assessment-toolkit.md` .

Security Assessment: View Findings / Drill Down

Field	Detail
Location	Same panel, results list.
Purpose	Review exactly what an active check found, with full supporting detail.
Role Required	**ADMIN, ANALYST, or VIEWER** (`security_assessment:read`).
Input	Click any finding row in a completed run's table.
Output	A dialog showing the finding's full title/description, a severity badge (info/low/medium/high/critical), any matched CVE IDs as badges, and the raw evidence JSON the severity was based on, pretty-printed.
Backend Process	Findings are already included in the `listSecurityAssessmentRuns()` response above; the dialog is purely a client-side detail view of already-fetched data.
Database Effect	None (read-only).
Error Conditions	None distinct from the run itself failing (in which case there are no findings to show).
Related Features	Run Security Assessment.

Event Log (Debug)

Field	Detail
Location	Right-hand sidebar, bottom-most card, labeled "Event log (debug)". Present on `/lookup/new` **only** (there is no live SSE stream on `/lookup/[id]`, so there's nothing to log there).
Purpose	A raw, timestamped list of every SSE event received so far (`detected` , each `provider_result` / `provider_summary` , `correlation` , `final_assessment` , `done` , or `error`) -- primarily useful for troubleshooting a stalled or misbehaving investigation.
Role Required	Same as the page.
Input	None (display only; scrollable).
Output	One line per event, oldest at top, newest at bottom, each with a wall-clock timestamp and a short label (e.g. `provider_result: virustotal (ok)`).
Backend Process	None of its own -- it mirrors the same event stream every other panel on the page consumes.
Database Effect	None.
Error Conditions	None.
Related Features	Start Investigation (SSE Stream).

Page: Completed Investigation (`/lookup/[id]`)

`app/lookup/[id]/page.tsx` renders **the exact same set of panels** documented above under `/lookup/new` -- same components, same props, same behavior -- fetched once via a single `getLookup(lookupId)` call (`GET /lookup/{id}`) instead of a live SSE stream. Rather than repeat every entry verbatim, this section documents only what's genuinely different here.

What	On `/lookup/new` (live)	On `/lookup/[id]` (completed)
Data source	SSE stream (`POST /lookup/stream`), events arrive incrementally	One `GET /lookup/{id}` call on mount; everything renders at once
AI Comparison Panel	Rendered once the investigation completes	**Not rendered at all** -- re-analyzing with a different AI backend is only offered from the live view
Event Log (debug)	Rendered in the sidebar	**Not rendered at all** -- there's no live stream to log
Sidebar footer	Nothing extra	A static line: "Loading static lookup data..." while fetching, then "Static lookup data loaded (no live SSE stream)." once done
Provider result shape	`ProviderResult` fields come directly off each SSE event	Reconstructed from the persisted `provider_results` table rows, which omit `ioc_value` / `ioc_type` / `fetched_at` / `from_cache` -- these are backfilled from the parent lookup record / sane defaults (`fetched_at: 0` , `from_cache: false`) so the same shared component still works unmodified
Correlation graph	Live `correlation` SSE event	Rebuilt server-side from persisted `correlation_edges` rows into the identical `{nodes, edges}` shape

Every other function -- Threat Score Gauge, Provider Cards, Final Assessment, Relationship Graph, MITRE Matrix, Detection Rules, Recommended Actions, all six Verdict Analysis modes, Show Receipts, Evidence Ledger, Pivot, Threat Hunting Center, Investigation Copilot, Add to Basket / Add to Case, Ask AI (Gemini), all four Export buttons, and the full Security Assessment panel -- works identically to its `/lookup/new` entry above, with identical role requirements, backend calls, database effects, and error conditions.

Load Completed Investigation

Field	Detail
Location	`/lookup/[id]` , on mount.
Purpose	Fetch a previously completed (or still- `pending` / `running` / `failed`) investigation's full record for display.
Role Required	**ADMIN, ANALYST, or VIEWER** (`lookup:read`).
Input	The `id` route parameter.
Output	Populates every panel described above; the header bar's status badge shows the lookup's real `status` (`pending` / `running` / `completed` / `failed`), and the post-completion panels (Verdict Analysis, Evidence, Pivot, Hunting Center, Copilot, Security Assessment) only render once `status === "completed"` .
Backend Process	`getLookup(lookupId)` → `GET /lookup/{id}` .
Database Effect	Read-only across `ioc_lookups` , `provider_results` , `ai_summaries` , `correlation_edges` , `final_assessment_records` .
Error Conditions	A fetch failure (e.g. the ID doesn't exist, or belongs to a lookup this deployment doesn't have) shows the caught error message in a red banner in place of the page content.
Related Features	Start Investigation (`/lookup/new`) -- the live counterpart of this same data.

Page: Executive Dashboard (`/dashboard`)

View Executive KPI Tiles

Field	Detail
Location	`/dashboard` , `app/dashboard/page.tsx` + `components/dashboard/KpiCard.tsx` -- seven tiles across the top of the page.
Purpose	A 30,000-foot, single-glance view of the platform's current state.
Role Required	**ADMIN, ANALYST, or VIEWER** (`dashboard:read` -- deliberately granted to all three roles, since this gates read-only operational visibility only, never anything credential-management or write-capable).
Input	None (display only).
Output	Seven tiles, each independently null-safe (a `null` value renders as "N/A", never coerced to 0, since for a metric like AI success rate, "no data" and "measured and zero" are different facts): **Active Investigations**; **Critical / High-Risk IOCs** (30-day window); **Open Cases**; **Open Critical Cases**; **Avg Threat Score** (30-day window, colored by the same risk-score banding as the Threat Score Gauge); **Provider Health** (percentage, 24-hour window); **AI Success Rate** (30-day window -- excludes `skipped_no_evidence` outcomes from both numerator and denominator, since correctly deciding not to call the AI at all is not a failure; shows "No AI activity in the last 30 days" as its caption when null rather than a bare "N/A" with no context).
Backend Process	`getKpis()` → `GET /dashboard/kpis` .
Database Effect	Read-only aggregate queries across `ioc_lookups` , `cases` , `provider_results` , and `final_assessment_records` .
Error Conditions	A fetch failure shows a red error banner above the tiles; the tiles themselves still render in their "N/A" state rather than disappearing.
Related Features	Provider Health page (the full version of the Provider Health tile); Cases page (the full version of the Open Cases tiles).

View Executive Summary

Field	Detail
Location	Same page, large card beneath the KPI row.
Purpose	An AI-written narrative layered on top of the exact same KPI numbers above -- or, if no AI backend is available, a real, number-accurate template narrative instead of a blank or broken card.
Role Required	**ADMIN, ANALYST, or VIEWER** (`dashboard:read`).
Input	None (display only).
Output	A paragraph of prose, with a badge disclosing whether it was **"AI-generated"** or a **"Template fallback"** -- an analyst should always know which one they're reading.
Backend Process	`getExecutiveSummary()` → `GET /dashboard/executive-summary` .
Database Effect	Read-only (same aggregates as the KPI tiles, plus one AI call if a backend is available and configured).
Error Conditions	If the endpoint 404s or otherwise fails, the card shows "Executive summary unavailable -- KPI tiles above still reflect live data." rather than a broken layout -- the KPI tiles' own success/failure is entirely independent of this card's.
Related Features	Executive KPI Tiles (the exact numbers this narrative is describing).

View Provider Health Widget (Dashboard)

Field	Detail
Location	Same page, card to the right of the Executive Summary.
Purpose	A compact, at-a-glance provider-health summary, with a link to the full page for detail.
Role Required	**ADMIN, ANALYST, or VIEWER** (`dashboard:read` -- the same permission gates `GET /providers/health` itself, not a separate one).
Input	Click "View full Provider Health status" to navigate to `/dashboard/provider-health` .
Output	A count per status (Healthy/Degraded/Down/Unknown, 24-hour window) with distinct icons and colors for each -- "Unknown" is never rendered as if it were "Healthy," since a provider with zero attempts in the window is a different fact from one that's been exercised and is fine. Below that, up to 5 non-healthy providers are listed by name with their status badge, or "All configured providers are healthy in the last 24h." if there are none.
Backend Process	`getProviderHealth()` → `GET /providers/health` .
Database Effect	Read-only against `provider_results` (aggregated per provider per time window).
Error Conditions	A fetch failure shows the caught error message in place of the widget's body.
Related Features	Provider Health page (`/dashboard/provider-health`), the full version of this same data.

Page: Provider Health (`/dashboard/provider-health`)

View Provider Health Table

Field	Detail
Location	`/dashboard/provider-health`, `app/dashboard/provider-health/page.tsx`.
Purpose	One row per IOC provider, showing whether it's actually working right now -- defaulting to the 24-hour window.
Role Required	**ADMIN, ANALYST, or VIEWER** (`dashboard:read`).
Input	None to view; click a row to expand (see next entry).
Output	Columns: provider name; category; whether it's configured (or "N/A (no key required)" for providers, like crt.sh or WHOIS/RDAP, that need no credential at all); a status badge (Healthy/Degraded/Down/Unknown); success rate; average latency; consecutive-failure count -- all for the 24h window. `success_rate` and `avg_latency_ms` render "N/A" (never "0%"/"0ms") when the provider had zero attempts in that window, since coercing a true data gap to zero would misrepresent it as "measured and found to be zero."
Backend Process	`getProviderHealth()` → `GET /providers/health`.
Database Effect	Read-only aggregate query against `provider_results`, grouped by provider and time window.
Error Conditions	A fetch failure shows a red error banner; "No providers configured" if the list is genuinely empty.
Related Features	Provider Health Widget (Dashboard); Provider Progress Tracker / Provider Result Cards (the per-investigation view of the same providers).

Expand Provider Detail (1h / 24h / 7d / 30d)

Field	Detail
Location	Same table, click any row.
Purpose	See all four measurement windows side by side for one provider, rather than just the default 24-hour column.
Role Required	Same as the table (`dashboard:read`).
Input	Click a provider row (toggles open/closed); only one row's detail is shown at a time.
Output	Four cards -- 1 Hour, 24 Hours, 7 Days, 30 Days -- each with its own status badge, success rate, average latency, consecutive failures, and rate-limited count, plus a line listing every IOC type that provider supports.
Backend Process	None additional -- the expanded detail is already present in the same `getProviderHealth()` response fetched for the table.
Database Effect	None additional (read-only).
Error Conditions	None distinct from the table's own fetch failure. A provider genuinely never exercised in a given window (e.g. a brand-new provider, or one whose IOC type simply hasn't come up in the last hour) correctly shows "Unknown" for that window specifically, even while other windows for the same provider show "Healthy" -- the four windows are independent, not rolled into one status.
Related Features	View Provider Health Table.

Page: IOC Basket (`/basket`)

Select / Deselect Basket Item

Field	Detail
Location	`/basket`, `app/basket/page.tsx`, checkbox on each row.
Purpose	Mark which collected IOCs the next bulk action (Investigate Selected / Compare Selected) should apply to.
Role Required	**ADMIN or ANALYST** to reach this page at all -- see **Error Conditions**.
Input	Click a row's checkbox.
Output	Toggles that item's membership in the current selection set (client-side state only).
Backend Process	None.
Database Effect	None.
Error Conditions	None of its own.
Related Features	Investigate Selected; Compare Selected (below).

Investigate Selected

Field	Detail
Location	Same page, toolbar.
Purpose	Re-run (or run for the first time) a lookup on every currently-selected Basket item, without re-typing each one.
Role Required	**ADMIN or ANALYST** (`lookup:create`, for the resulting investigations; reaching this page's data at all already required `basket:manage`).
Input	Select one or more items; click "Investigate Selected" (disabled with none selected).
Output	Opens one new browser tab per selected item, each navigating to `/lookup/new?value=<that item's IOC>`.
Backend Process	None directly -- `window.open()` per item; the actual investigations run on their own new tabs (see Start Investigation).
Database Effect	None from this click itself; each resulting tab creates its own `ioc_lookups` row once its investigation runs.
Error Conditions	Browser popup-blocking could prevent some tabs from opening if many items are selected at once -- no in-app error is shown for this, since it's outside the page's own control.
Related Features	Start Investigation.

Compare Selected

Field	Detail
Location	Same page, toolbar.
Purpose	View two or more collected IOCs side by side -- verdict, risk score, ASN, malware families, threat actors -- to check whether tracked indicators relate to each other, plus an AI-written comparison narrative.
Role Required	**ADMIN or ANALYST**.
Input	Select at least 2 items that already have a completed lookup (`latest_lookup_id` set); click "Compare Selected".
Output	A comparison table (one row per IOC) with the platform's assessment of the single most dangerous one highlighted, plus a narrative paragraph and a bulleted list of key differences.
Backend Process	`compareBasketIOCs(lookupIds)` → `POST /basket/compare` .
Database Effect	Read-only against `ioc_lookups` / `final_assessment_records` / `correlation_edges` for the selected lookups; no new rows written.
Error Conditions	Selecting fewer than 2 basket items with a completed lookup shows "Select at least 2 basket items that already have a completed lookup to compare." without calling the backend at all; a backend failure surfaces its `detail` message inline.
Related Features	Investigate Selected; Case comparison is not a separate feature -- this is the only cross-IOC comparison tool in the platform.

Clear Basket

Field	Detail
Location	Same page, toolbar.
Purpose	Empty the entire personal scratch-list at once.
Role Required	**ADMIN or ANALYST** (`basket:manage`).
Input	Click "Clear Basket" (disabled if the Basket is already empty).
Output	The list empties immediately, with no confirmation dialog.
Backend Process	`clearBasket()` → `DELETE /basket` .
Database Effect	Deletes every row in `basket_items` belonging to the current user.
Error Conditions	None surfaced distinctly -- a failure would leave the (now-stale) list displayed, since the client optimistically clears local state without re-checking the response.
Related Features	Remove Single Item (a scoped-down version of the same action).

Remove Single Item

Field	Detail
Location	Same page, trash icon on each row.
Purpose	Remove one specific IOC from the Basket without clearing everything.
Role Required	**ADMIN or ANALYST** (`basket:manage`).
Input	Click the trash icon on a row.
Output	That row disappears from the list immediately; if it was selected, it's also removed from the selection set.
Backend Process	`removeFromBasket(itemId)` → `DELETE /basket/{item_id}` .
Database Effect	Deletes one row from `basket_items` .
Error Conditions	None surfaced distinctly (same optimistic-update caveat as Clear Basket).
Related Features	Clear Basket.

Investigate Single Item

Field	Detail
Location	Same page, "Investigate" button on each row.
Purpose	Jump straight from one Basket entry into a fresh investigation of it.
Role Required	**ADMIN or ANALYST** (`lookup:create`).
Input	Click "Investigate" on a row.
Output	Navigates to `/lookup/new?value=<that item's IOC>` in the current tab (unlike the bulk "Investigate Selected," which opens new tabs).
Backend Process	None directly -- navigation only.
Database Effect	None from this click itself.
Error Conditions	None distinct from Start Investigation's own.
Related Features	Investigate Selected; Start Investigation.

Page-level note -- Role Required: every function on the Basket page, including simply *loading* it, calls `GET /basket` first, which requires `basket:manage` . That permission is granted to **ADMIN and ANALYST only** -- **VIEWER does not have it.** A VIEWER who navigates to `/basket` (nothing in the frontend prevents this; the Global Navigation Bar's Basket link is visible to every logged-in role) will see the page shell render, but the item list itself fails to load (the caught error is shown inline: "Failed to fetch"/the backend's `403` detail), and every action button is effectively non-functional for that role. This is a deliberate exception to the platform's usual "VIEWER gets broad read-only visibility" pattern -- the Basket is treated as a personal analyst workspace, not a read-only operational view.

Page: Cases (`/cases`)

View Case List

Field	Detail
Location	`/cases`, `app/cases/page.tsx`.
Purpose	See every case on the platform, with its severity and status at a glance.
Role Required	**ADMIN, ANALYST, or VIEWER** (`case:read`).
Input	Click any case row.
Output	A list sorted by creation date, each row showing title, creation timestamp, tags (if any), a severity badge (low/medium/high/critical), and a status badge (open/investigating/contained/resolved/false_positive/closed). Clicking a row navigates to `/cases/{id}`.
Backend Process	`listCases()` → `GET /cases`.
Database Effect	Read-only against `cases`.
Error Conditions	Fetch failure shows a red error banner; "No cases yet." if the list is genuinely empty.
Related Features	Case Detail page; New Case (below).

New Case / Create Case

Field	Detail
Location	Same page, "New Case" button toggles an inline form.
Purpose	Start a new case container to group related IOCs and analyst notes for one incident.
Role Required	**ADMIN or ANALYST** (`case:create`) -- the "New Case" button and form are shown to every role that can reach this page, including VIEWER; see **Error Conditions**.
Input	Title (required); description (optional); severity (low/medium/high/critical, defaults to medium).
Output	On success, navigates directly to the new case's detail page (`/cases/{id}`).
Backend Process	`createCase(title, description, severity)` → `POST /cases`.
Database Effect	Inserts one row into `cases`.
Error Conditions	The submit button is disabled while the title is blank. A **VIEWER** can open the form and fill it in, but submitting fails server-side with `403` (`case:create` excludes VIEWER) -- shown inline as "Failed to create case." beneath the form; the form itself is not hidden from a VIEWER anywhere in this page's code.
Related Features	Case Detail page; Add to Case (from any investigation's Investigation Actions).

Page: Case Detail (`/cases/[id]`)

View Case Detail

Field	Detail
Location	`/cases/[id]`, `app/cases/[id]/page.tsx`.
Purpose	The single record for one incident: its metadata, every attached IOC, and every analyst note.
Role Required	**ADMIN, ANALYST, or VIEWER** (`case:read`).
Input	None to view.
Output	Title, description, severity/tag pills; a status dropdown (see below); an "IOCs (N)" card listing each attached IOC with an "Investigate" shortcut; an "Analyst Notes (N)" card listing every note with its timestamp, plus a note-entry form.
Backend Process	`getCase(caseId)` → `GET /cases/{id}`.
Database Effect	Read-only against `cases`, `case_iocs`, `case_notes` (and `case_reports`, though the `CaseDetail` response's `reports` array is not currently rendered anywhere on this page -- the data model supports case reports, but no report-generation or report-viewing UI is wired up to it yet).
Error Conditions	A fetch failure replaces the whole page body with a red error banner.
Related Features	Case list; Add to Case (from any investigation).

Change Case Status

Field	Detail
Location	Same page, dropdown next to the title.
Purpose	Move a case through its lifecycle: open → investigating → contained/resolved/false_positive → closed.
Role Required	**ADMIN or ANALYST** (`case:write`). Note: the backend also defines a separate `POST /cases/{id}/close` endpoint gated by a distinct `case:close` permission, but the current UI does **not** use it -- selecting "closed" from this same dropdown goes through the generic status-update call below, which only requires `case:write`, not `case:close`.
Input	Select a new status from the dropdown.
Output	The case reloads with the new status reflected immediately.
Backend Process	`updateCase(caseId, { status })` → `PATCH /cases/{id}`.
Database Effect	Updates the `status` column on the matching `cases` row.
Error Conditions	A VIEWER (who can view but not write) would get `403` on this call, though the dropdown is not hidden from a VIEWER's view of the page -- no inline error handling is wired up for this specific call in the current page code, so a failure here would not show a visible message (a real, current gap, not an invented one).
Related Features	View Case Detail.

Investigate IOC (from Case)

Field	Detail
Location	Same page, "Investigate" button on each attached IOC row.
Purpose	Jump from a case's attached IOC straight into a fresh investigation of it.
Role Required	**ADMIN or ANALYST** (`lookup:create`) for the resulting investigation; viewing the case itself only needs `case:read` .
Input	Click "Investigate" on an IOC row.
Output	Navigates to `/lookup/new?value=<that IOC>` .
Backend Process	None directly -- navigation only.
Database Effect	None from this click itself.
Error Conditions	None distinct from Start Investigation's own (including the same VIEWER-gets-403-on-the-destination behavior).
Related Features	Add to Case (the reverse direction -- attaching an IOC found elsewhere to this case); note that the backend also defines `DELETE /cases/{case_id}/iocs/{ioc_id}` (`case:write`) to detach an IOC, but this page currently offers no button to do so -- a real, current gap.

Add Analyst Note

Field	Detail
Location	Same page, bottom of the Analyst Notes card.
Purpose	Record a freeform observation that doesn't belong inside an automated provider result -- e.g. "confirmed this CVE affects our externally-facing logging service; escalating to patch team."
Role Required	**ADMIN or ANALYST** (`case:write`).
Input	Free-text note body.
Output	The new note appears at the top of the notes list with its timestamp; the input clears.
Backend Process	`addCaseNote(caseId, body)` → `POST /cases/{id}/notes` (the underlying function also accepts an optional `anchorType` / `anchorRef` pair to link a note to a specific piece of evidence, but the Case Detail page's current note form doesn't set either -- every note added from this UI is unanchored).
Database Effect	Inserts one row into `case_notes` .
Error Conditions	The submit button is disabled while the note text is blank; a failure (e.g. a VIEWER somehow reaching this call) shows the caught error message inline.
Related Features	View Case Detail.

Page: Manage Providers (`/providers`)

Page-level note -- Role Required: this page has **no client-side role gate at all** -- unlike `/admin`, any logged-in user can navigate here and the Global Navigation Bar's "Providers" link is visible to every role. However, **every single API call this page makes requires the `provider:manage` permission, which only ADMIN holds** (listing AI providers, listing IOC providers, fetching the audit log which needs `audit:read` -- also ADMIN-only -- configuring, testing, enabling/disabling, and setting-active). Every one of those calls on this page is wrapped in a `.catch(() => {})` that silently swallows the error. The practical, observable result: an ANALYST or VIEWER who visits `/providers` sees the page's tabs and layout render, but every tab's content stays empty (no providers listed, no audit log entries) with no error message at all -- not a `403` banner, not a redirect, just an empty-looking page. This is a real, current gap between "who can see this page" and "who can use it," documented here rather than glossed over.

AI Providers: Expand / Edit Credentials

Field	Detail
Location	`/providers`, AI Providers tab, `components/dashboard/ProviderConfigRow.tsx`.
Purpose	Enter or update the credentials for one AI backend (Ollama, Anthropic/Claude, AWS Bedrock, Gemini, Groq).
Role Required	**ADMIN** (`provider:manage`).
Input	Click a provider's row to expand it; type into the credential field(s) specific to that backend (e.g. `api_key` for Anthropic/Gemini/Groq; `base_url` for Ollama; `bedrock_api_key` / `aws_access_key_id` / `aws_secret_access_key` / `aws_region` for Bedrock) and, if applicable, a model ID.
Output	An expanded form with password-masked input fields, pre-filled with a masked placeholder (e.g. `sk-...ab12`) if a credential is already saved -- never the real stored value.
Backend Process	None yet -- this only opens the form (see Save Configuration, below, for the actual write).
Database Effect	None from expanding alone.
Error Conditions	None.
Related Features	Save Configuration; Test Connection; Set Active Backend.

AI Providers: Test Connection

Field	Detail
Location	Same row, "Test Connection" button.
Purpose	Verify a candidate credential actually works against the real backend before saving it.
Role Required	**ADMIN** (`provider:manage`).
Input	Whatever is currently typed into the credential field(s) (not the already-saved, masked value -- the backend's test endpoint only ever receives a freshly-typed candidate).
Output	"OK: <message>" or "Failed: <message>" beneath the form, including latency where applicable; this result, plus any prior test's timestamp/outcome, is also shown as "Last test: OK/Failed -- <message>" even after the form is collapsed and reopened.
Backend Process	`testAIBackend(backend, credentials, model)` → `POST /ai/test` ; the result is then persisted via `recordAITestResult(backend, ok, message)` → `POST /runtime/ai-providers/{backend}/record-test` .
Database Effect	Updates `last_test_at` / `last_test_ok` / `last_test_message` on the matching `provider_runtime_configs` row; writes one entry to `config_audit_log` . Does **not** persist the tested credential itself unless Save is also clicked.
Error Conditions	A network failure or invalid credential shows "Failed: <the real error/rejection reason>" -- never a generic "something went wrong."
Related Features	Save Configuration; IOC Providers: Test Connection (the same pattern, different endpoint).

AI Providers: Save Configuration

Field	Detail
Location	Same row, "Save" button.
Purpose	Persist a new or updated credential/model for one AI backend, effective immediately with no restart.
Role Required	**ADMIN** (`provider:manage`).
Input	Whatever is currently typed into the credential/model fields.
Output	The row's "Configured" badge updates immediately; the credential fields clear back to their masked-placeholder state.
Backend Process	`configureAIProvider(backend, credentials, modelId)` → `POST /runtime/ai-providers/{backend}` .
Database Effect	Upserts the matching row in `provider_runtime_configs` (credentials stored encrypted, never in plaintext); writes one entry to `config_audit_log` describing the change without ever including the credential value itself.
Error Conditions	A save failure shows the backend's `detail` message in the page-level error banner.
Related Features	Test Connection; Set Active Backend; AI Quick Switch (home page).

AI Providers: Set Active Backend

Field	Detail
Location	Same row, "Set Active" button (hidden once a provider is already the active one).
Purpose	Make this backend the one used for every *new* investigation platform-wide, starting immediately.
Role Required	**ADMIN** (`provider:manage`).
Input	Click "Set Active".
Output	This row gains an "Active" badge; any other row's "Active" badge disappears.
Backend Process	`setActiveAIBackend(backend)` → `POST /runtime/ai-active` .
Database Effect	Updates the `is_active` flag across `provider_runtime_configs` rows of kind `ai` ; writes one entry to `config_audit_log` .
Error Conditions	Failure shows the backend's `detail` message in the page-level error banner.
Related Features	AI Quick Switch (the same action, exposed as a shortcut on the home page).

IOC Providers: Expand / Edit Credentials

Field	Detail
Location	`/providers` , IOC Providers tab, same `ProviderConfigRow` component reused for all 18 registered IOC providers.
Purpose	Enter or update API credentials for a threat-intelligence/OSINT provider (for the providers that need one -- crt.sh, NVD, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, PhishTank, and the Internet Intelligence Collector need none at all, and show "This provider needs no API key." instead of a credential form).
Role Required	**ADMIN** (`provider:manage`).
Input	Click a row to expand; type into whichever credential field(s) that specific provider declares (`credential_fields` , provider-specific -- e.g. VirusTotal needs one API key, Censys needs an API ID and secret).
Output	Same expand/masked-placeholder behavior as AI Providers.
Backend Process	None yet (form only).
Database Effect	None from expanding alone.
Error Conditions	None.
Related Features	IOC Providers: Save Configuration, Test Connection, Enable/Disable.

IOC Providers: Test Connection

Field	Detail
Location	Same row.
Purpose	Verify a candidate IOC-provider credential works before saving.
Role Required	**ADMIN** (`provider:manage`).
Input	Currently-typed credential field(s).
Output	Same OK/Failed result pattern as AI Providers' Test Connection.
Backend Process	`testIOCProvider(providerId, credentials)` → `POST /providers/{provider_id}/test` ; result persisted via `recordIOCTestResult(providerId, ok, message)` → `POST /runtime/ioc-providers/{provider_id}/record-test` .
Database Effect	Updates the matching `provider_runtime_configs` row's last-test fields; writes to `config_audit_log` .
Error Conditions	Same pattern as AI Providers' Test Connection.
Related Features	IOC Providers: Save Configuration.

IOC Providers: Save Configuration

Field	Detail
Location	Same row, "Save" button.
Purpose	Persist a new/updated credential for one IOC provider, effective on the very next investigation, no restart.
Role Required	**ADMIN** (`provider:manage`).
Input	Currently-typed credential field(s).
Output	"Configured" badge updates; fields clear back to masked placeholders.
Backend Process	`configureIOCProvider(providerId, credentials)` → `POST /runtime/ioc-providers/{provider_id}` .
Database Effect	Upserts the matching `provider_runtime_configs` row (kind `ioc`); writes to `config_audit_log` .
Error Conditions	Failure shows the backend's `detail` message in the page-level error banner.
Related Features	IOC Providers: Enable/Disable.

IOC Providers: Enable / Disable

Field	Detail
Location	Same row, "Enable"/"Disable" button (this control has no AI-Providers equivalent -- AI backends are switched via "Set Active" instead, since exactly one is active at a time, whereas any number of IOC providers can be independently enabled).
Purpose	Turn a specific IOC provider on or off for future investigations without removing its saved credentials.
Role Required	**ADMIN** (`provider:manage`).
Input	Click the toggle.
Output	The row's "Enabled"/"Disabled" badge flips immediately.
Backend Process	`setIOCProviderEnabled(providerId, enabled)` → `POST /runtime/ioc-providers/{provider_id}/enabled` .
Database Effect	Updates the `enabled` flag on the matching `provider_runtime_configs` row; writes to `config_audit_log` .
Error Conditions	Failure shows the backend's `detail` message in the page-level error banner.
Related Features	IOC Providers: Save Configuration; Provider Progress Tracker (a disabled provider is never queried, so it never appears in an investigation's results at all -- distinct from a configured-but-erroring provider, which does appear, with an "Error" status).

View Configuration Audit Log (Providers Page)

Field	Detail
Location	`/providers` , Audit Log tab. Renders the same list/rendering block as the Administration console's own Audit Log tab -- one shared table, not a duplicate implementation.
Purpose	Review every configuration change ever made through this page (or the Administration console), without ever exposing a credential value.
Role Required	**ADMIN** (`audit:read`).
Input	None (display only).
Output	A chronological list: timestamp, action, and a detail string, e.g. "Configured AI provider `anthropic` " -- never the credential itself.
Backend Process	`getAuditLog(50)` → `GET /runtime/audit-log?limit=50` .
Database Effect	Read-only against `config_audit_log` .
Error Conditions	"No changes recorded yet." if empty. Since this whole page's calls silently fail for non-admins (see the page-level note above), a non-admin sees this tab permanently empty rather than an explicit access-denied message.
Related Features	Administration → Audit Log (the identical table, reached from a different page).

View Network Access / Copy LAN URL

Field	Detail
Location	`/providers`, Network Access tab, `components/dashboard/NetworkAccessPanel.tsx`.
Purpose	Show how to reach this HORIZON GRID instance from another device on the same network -- useful for a small team sharing one installation.
Role Required	None beyond being logged in -- this is the one tab on this page that calls an **unauthenticated** backend endpoint, so it works even for a role that can't use anything else on this page.
Input	Click "Copy LAN URL" (only shown once a LAN IP has actually been detected).
Output	Shows "This computer" (the browser's own current origin), "From another device on this network" (the detected LAN IP and frontend port, e.g. `http://192.168.1.42:3000`), and the backend port. Copying writes the LAN URL to the clipboard with a "Copied." confirmation.
Backend Process	`getNetworkInfo()` → `GET /network-info` (unauthenticated -- the same trust model as the backend's own `/health` endpoint).
Database Effect	None.
Error Conditions	"Not detected. Run 'Configuration' from the Start Menu to detect it." if no LAN IP was ever recorded during setup; "Could not reach the backend for network info." if the call itself fails. If clipboard access is denied, the message asks the user to copy manually instead.
Related Features	None -- this is a standalone informational tool tied to the Windows installer's Setup Wizard/Configuration tool, not to any other in-app feature.

Page: Administration (`/admin`)

Page-level note -- Role Required: unlike `/providers`, this page **does** guard itself client-side: it checks `isLoggedIn()` first (redirect to `/login`), then `getCurrentUser().role !== "admin"` (redirect all the way to `/`, not just an error message). Every mutating action on this page is additionally, independently enforced server-side by `require_permission("user:manage")` on every `/api/v1/admin/*` route -- the client-side check exists purely so a non-admin never sees a screen full of buttons that would all `403`, not as the actual security boundary.

View Overview

Field	Detail
Location	`/admin`, Overview tab (default).
Purpose	A quick account-health snapshot: how many users exist, how many are active/disabled, how many are admins, and who's logged in most recently.
Role Required	**ADMIN** (`user:manage`).
Input	None (display only).
Output	Four stat cards (Total Users, Active, Disabled, Admins) plus a "Recent Logins" list (email + timestamp, most recent first).
Backend Process	`getUserStats()` → `GET /admin/users/stats`.
Database Effect	Read-only aggregate query against `users`.
Error Conditions	"No logins recorded yet." if no user has ever logged in.
Related Features	Users tab (the full, filterable list behind these counts).

Users: Search / Filter / Sort / Paginate

Field	Detail
Location	`/admin`, Users tab, `components/dashboard/UsersManagementPanel.tsx`.
Purpose	Find a specific user account among potentially many.
Role Required	**ADMIN** (`user:manage`).
Input	A search string (matched against email or name); a role filter (admin/analyst/viewer/any); an active-status filter (active/disabled/any); click a sortable column header (Email, Role, Last Login, Created) to sort/reverse-sort by it; Previous/Next pagination (25 per page).
Output	The table re-fetches and re-renders on every filter/sort/page change.
Backend Process	`listUsers({ search, role, isActive, page, pageSize, sortBy, sortDir })` → `GET /admin/users?...`.
Database Effect	Read-only, filtered/sorted/paginated query against `users`.
Error Conditions	"No users match these filters." if a filter combination returns nothing; a fetch failure shows an inline error banner.
Related Features	View Overview; Create User; Edit User.

Users: Create User

Field	Detail
Location	Same tab, "+ New User" button opens a dialog.
Purpose	The only way to add an account after the very first (bootstrap) account has been created -- self-registration is permanently closed at that point.
Role Required	**ADMIN** (`user:manage`).
Input	Full name (optional); email (required); initial password (required, minimum 8 characters); role (admin/analyst/viewer, defaults to analyst).
Output	The dialog explicitly states: "The account is active immediately -- no email verification exists." On success, the dialog closes and the user list refreshes to include the new account.
Backend Process	`createUser(email, password, fullName, role)` → `POST /admin/users` .
Database Effect	Inserts one row into `users` .
Error Conditions	A duplicate email or other validation failure shows the backend's `detail` message inline in the dialog, not as a page-level banner.
Related Features	Create Account (`/register` , bootstrap-only); Edit User.

Users: Edit User

Field	Detail
Location	Same tab, "Edit" button per row, opens a dialog.
Purpose	Change a user's display name or role.
Role Required	**ADMIN** (`user:manage`).
Input	Full name; role (disabled -- cannot be changed -- if editing your own account: "*You can't change your own role -- ask another administrator.*").
Output	Dialog closes; list refreshes with the updated values.
Backend Process	`updateUser(userId, { full_name, role })` → `PATCH /admin/users/{id}` .
Database Effect	Updates `full_name` / `role` on the matching `users` row.
Error Conditions	The backend independently rejects an admin's attempt to change their own role (a last-admin/self-demotion safeguard) even if the client-side disabling were somehow bypassed; the error surfaces inline in the dialog.
Related Features	Create User; View Roles & Permissions (what each role actually grants).

Users: Reset Password

Field	Detail
Location	Same tab, "Reset Password" button per row, opens a dialog.
Purpose	Let an administrator set a new password for a user who's lost access to their own -- there is no self-service "forgot password" flow anywhere in the product, so this is the only recovery path.
Role Required	**ADMIN** (`user:manage`).
Input	New password (minimum 8 characters); confirm new password (must match).
Output	The dialog states plainly: "*This immediately signs the user out of every existing session -- they'll need to sign in again with the new password.*"
Backend Process	`resetUserPassword(userId, newPassword)` → `POST /admin/users/{id}/reset-password` .
Database Effect	Updates `hashed_password` on the matching `users` row and increments its `token_version` counter -- which is embedded in and re-checked against every issued JWT, so this single write invalidates every access/refresh token that user currently holds, not just future logins.
Error Conditions	Client-side: submitting is blocked while the two password fields don't match ("Passwords do not match.") or the password is under 8 characters. Server-side failures surface inline in the dialog.
Related Features	Users: Enable/Disable Account (the other administrator-initiated access-control action).

Users: Enable / Disable Account

Field	Detail
Location	Same tab, a role-colored button per row ("Disable" for an active account, "Enable" for a disabled one), opens a confirmation dialog.
Purpose	Immediately grant or revoke a user's access without deleting their account or history.
Role Required	**ADMIN** (`user:manage`).
Input	Confirm in the dialog.
Output	Disabling states: "*They will immediately lose access -- any current session stops working on their very next request.*" Enabling states: "*They will regain access immediately, with no restart required.*"
Backend Process	`setUserActive(userId, isActive)` → `POST /admin/users/{id}/active` .
Database Effect	Updates the `is_active` flag on the matching `users` row.
Error Conditions	A safeguard against disabling the last remaining active admin (preventing total lockout) is enforced server-side; its rejection surfaces inline in the dialog.
Related Features	Users: Reset Password.

View Roles & Permissions

Field	Detail
Location	`/admin`, Roles & Permissions tab.
Purpose	A read-only reference so an administrator can see exactly what each of the three roles can do, without having to read source code.
Role Required	**ADMIN** (`user:manage`).
Input	None (display only -- the page states plainly that roles/permissions are fixed in code and not editable here).
Output	One card per role (Admin, Analyst, Viewer), each listing its full permission set as badges.
Backend Process	`getRoles()` → `GET /admin/roles`.
Database Effect	None (the permission matrix is a static in-code dictionary, `ROLE_PERMISSIONS` in `backend/app/models/user.py`, not a database table -- this endpoint reflects that fixed dictionary, not a queryable/editable record).
Error Conditions	None distinct from a general fetch failure.
Related Features	Every "Role Required" field in this entire chapter is drawn from this same fixed matrix.

View Audit Log (Administration)

Field	Detail
Location	`/admin`, Audit Log tab -- the identical rendering block as `/providers` ' own Audit Log tab (one shared table, reached from two places).
Purpose	Review every account and configuration change together in one place, without ever exposing a credential or password value.
Role Required	**ADMIN** (`audit:read`).
Input	None (display only).
Output	Chronological list of every action: user creation, edits, password resets, enable/disable, alongside provider/AI configuration changes made from `/providers` -- one unified log, not two separate ones.
Backend Process	`getAuditLog(50)` → `GET /runtime/audit-log?limit=50`.
Database Effect	Read-only against `config_audit_log`.
Error Conditions	"No changes recorded yet." if empty.
Related Features	Manage Providers → View Configuration Audit Log (the same table).